

Section 1 — Presentation Script

System Architecture, Technology Decisions & Project Vision

Presenter: Ahmed Tawfik — Team Leader & System Architect **Estimated time:** 14–16 minutes **Tone:** Confident, collaborative, professional, visionary

Before you step up: You led the architectural design and engineering of this entire project. You made the structural decisions that allowed every other subsection to be built cleanly and independently. While presenting, own the vision — not with arrogance, but with the quiet confidence of someone who genuinely understands every component. Emphasize collaboration: your architectural choices *empowered* your team. Speak slowly on technical points. Pause after visuals to let the committee absorb.

قبل ما تقف تقدم: انت اللي صممت ال architecture بتاع ال project ده كله. القرارات اللي اتخذت خلّت كل حد في ال team يشتغل بثقة من غير ما يتعارض مع حد. وانت بتقدم، اتكلم بثقة — مش غرور، الثقة اللي بييجي من إنك فاهم كل حاجة جوا ال system. اتكلم ببطء على النقاط التقنية. وسبب pause بعد كل visual عشان اللجنة تاخذ وقتها.

SLIDE 1 — Title Slide

Title: Cortex — A Bilingual AI-Powered Academic Workspace **Subtitle:** Architecture, Vision & Engineering Decisions **Your name:** Ahmed Tawfik — Team Leader & System Architect

Speaker notes: "Good morning. My name is Ahmed Tawfik — team leader and system architect for Cortex.

What you will see today is not a collection of loosely related features. It is a unified engineering system, designed from the ground up with a single guiding philosophy: every tool a university student needs — for studying, organizing, collaborating, and focusing — should work together as one coherent platform.

My role was to design that platform: the architecture, the technology choices, the data layer, and the system boundaries that allowed my teammates to each build their

subsystem with confidence, knowing it would integrate perfectly with the rest.

I will begin by explaining the problem that motivated this project. Then I will walk you through the product we built, the architecture behind it, and the reasoning behind every technology decision we made."

(Pause. Look at the committee. Then:) "Let me start with the reality of being a university student in Egypt today."

🇪🇬 team leader و system architect ل Cortex. «بالعربي: صباح الخير. أنا أحمد توفيق

اللي هتشوفوه النهارده مش مجموعة features متجمّعة مع بعض. ده system هندسي متكامل، اتصمّم من الصفر بفكرة واحدة: كل tool المذاكر محتاجه — من ناحية المذاكرة، والتنظيم، والتعاون، وال focus — المفروض يشتغلوا مع بعض ك platform واحد متماسك.

دوري كان إني أصمم ال platform ده: ال architecture، اختيارات التكنولوجيا، ال data layer، والحدود اللي بين ال systems — اللي خلّت كل زميل في ال team يبني ال subsystem بتاعه بثقة، عارف إنه هيتكامل مع الباقي بشكل صح.

هبدأ بشرح المشكلة اللي خلتنا نعمل ال project ده. وبعدين هأخذكُم في جولة على ال product اللي بنيناه، وال architecture وراه، وسبب كل قرار تكنولوجي اتخد.»

(وقفة. بص على اللجنة. وبعدين:) «خليني أبدأ بالواقع اللي بيعيشه أي طالب جامعي في مصر النهارده.»

Timing: 45 seconds

SLIDE 2 — The Problem: Five Apps, Zero Integration

Title: The Egyptian University Student's Reality

Visual — Five disconnected islands:

Google Docs	WhatsApp	Phone Timer	Calendar	Chat
Your notes	Materials	Pomodoro	Schedule	Quest
(no AI)	(expires)	(no context)	(no notes)	(no t
[nothing]	[nothing]	[nothing]	[nothing]	[noti

Bullets:

- Notes in Google Docs — no structure, no AI, disconnected from courses
- Course materials in WhatsApp — unsearchable, expires, unorganized
- Study timer on a phone app that knows nothing about what you are studying
- ChatGPT that answers from the internet, not from what YOU actually studied
- No tool designed specifically for Arabic-speaking Egyptian university students

Speaker notes: "This is the reality of every Egyptian university student today. And I do not mean 'some students' — I mean every single one of us.

Your lecture notes live in Google Docs. Your professor's slides arrive in a WhatsApp group that, after one semester, has 3,000 messages and is completely unsearchable. Your study timer is a phone app. Your calendar is another app. Your AI assistant is ChatGPT — which has never read a single one of your notes.

These five tools do not share a single byte of information. Every time you switch between them, you lose context. You lose flow. You lose time.

We lived this exact frustration as students at Menoufia University. And our team decided: instead of finding a slightly better version of one of these tools, we would build a unified system that replaces all five — designed specifically for us, for Egyptian students, in Arabic and English from day one.

That system is Cortex."

بالعربي: «ده هو الواقع اللي بيعيشه كل طالب جامعي في مصر النهارده. ومش بقول "بعض الطلبة" — بقول كل واحد فينا بدون استثناء

ال notes بتاعتك موجودة في Google Docs. ال slides بتيجي في WhatsApp group اللي بعد ترم واحد فيه 3000 رسالة ومحدث لاقى حاجة فيه. ال timer على التليفون. ال calendar app تاني. وال AI assistant هو ChatGPT — اللي ما قراش ولا note واحدة منك في حياته.

ال tools دي الخمسة مش بتشارك ولا byte واحدة من المعلومات مع بعض. كل مرة تعدي من واحدة للتانية بتخسر ال context، بتخسر ال flow، وبتخسر وقت.

احنا عشنا الإحساس ده بالضبط كطلبة في جامعة المنوفية. وقررنا ك team: بدل ما نلاقي نسخة أحسن شوية من حاجة موجودة، نبني system متكامل يحل الخمسة مشاكل دول — مصمم لينا خصيصاً، للطلبة المصريين، بالعربي والإنجليزي من أول يوم.

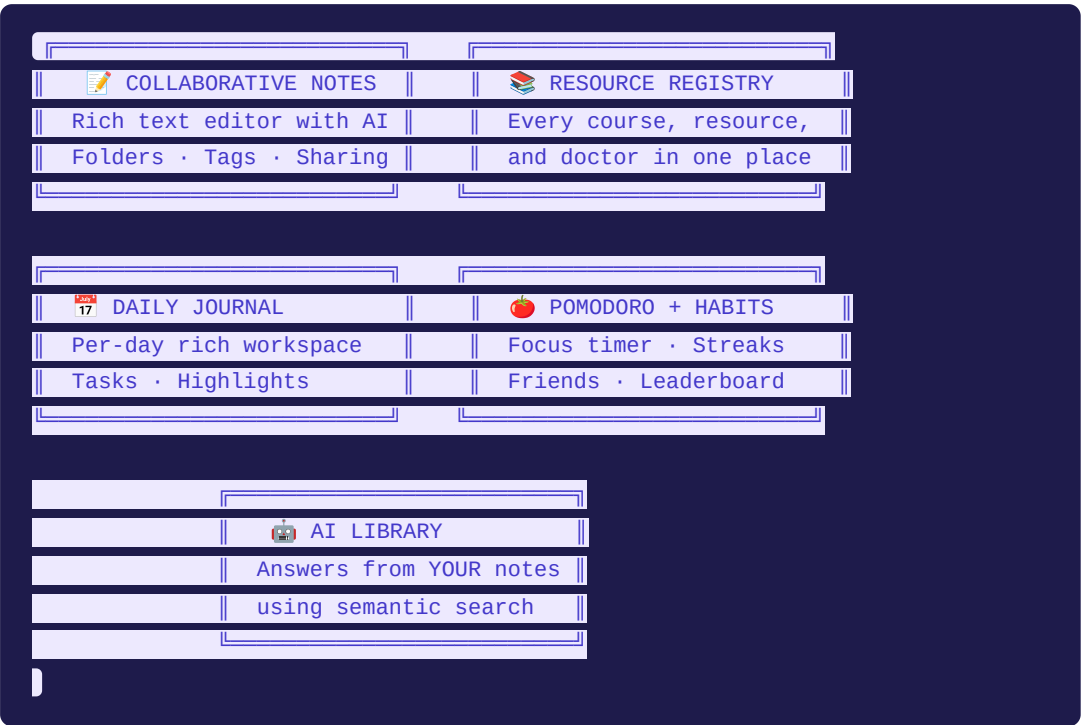
ال system ده هو Cortex.»

Timing: 1 min 30 sec **Transition:** "Let me show you what we built."

SLIDE 3 — What Is Cortex? — Five Integrated Systems

Title: One Platform. Five Complete Systems.

Visual — Five tiles, each a standalone product:



Speaker notes: "Cortex has five integrated systems. And I want to be precise: these are not five features — they are five complete products, each of which could stand alone as a standalone application. We built all five.

The Notes system is a full rich-text workspace with an embedded AI assistant — not a text file. You write structured documents with headings, tables, code blocks, images.

The Resource Registry is a structured university catalog, not a shared folder. Every course, resource, and doctor at your university, organized hierarchically and pre-filtered to your curriculum.

The Daily Journal is a calendar-based reflective workspace with tasks and a highlight field — not a to-do list. It builds a searchable record of your academic progress over the entire semester.

The Pomodoro system includes habits, streaks, a social friend system, and a weekly leaderboard — not just a timer.

And tying everything together: the AI Library Assistant. It can answer questions from what the student has actually written in their notes and daily logs. Not from the internet. From their own library.

And there are two more systems I have not mentioned yet."

دول — features متكاملة. وعازيز أكون دقيق: مش خمس systems فيه خمس Cortex: بالعربي 🇪🇬 واحنا بنيناهم الخمسة. application كاملة، كل واحدة ممكن تقف لوحدها ك products خمس

ال Notes System ده بيئة كتابة rich text كاملة مع AI مدمج جواه — مش text file عادي. بتكتب documents منظمة: headings, tables, code blocks, صور.

ال Resource Registry ده catalog منظم للجامعة — مش shared folder. كل course، كل material، وكل دكتور في جامعتك، منظمين بشكل هرمي ومفلترين ل curriculum بتاعك تلقائياً. ال Daily Journal ده workspace لكل يوم فيه مكان لل tasks وحقل ال highlight — مش مجرد to-do list. بيبيني سجل قابل للبحث لمسيرتك الدراسية طول الترم.

ال Pomodoro system فيه streaks, habits, نظام اجتماعي مع أصحابك، و weekly leaderboard — مش مجرد timer.

واللي بيربط كل ده مع بعض: ال AI Library Assistant. ممكن يجاوب على أسئلتك من اللي انت كتبتة بنفسك في ال notes وال daily logs. مش من الإنترنت — من مكتبتك الشخصية.

وفي اتنين systems تانيين لسه ما ذكرتهمش.»

Timing: 1 min 45 sec **Transition:** "Cortex does not just live in a browser tab."

SLIDE 4 — Beyond the Browser: Two Native Extensions

Title: The Chrome Extension + The GNOME Extension (Tamatem)

Visual — Side-by-side comparison:

CHROME EXTENSION	GNOME EXTENSION — Tamatem
(Manifest V3)	(tamatem@frey.dev · GNOME Shell 48-50)
[🍅 Timer] [🚫 Blocker] [👥 Social]	
[📊 History] [⚙️ Settings]	🍅 OS Algorithms — 19:44 [■]
• Timer runs in service worker (no tab required)	• Timer lives in Linux system tray (no browser required)
• Blocks sites at HTTP request level	• Blocks sites at OS-network level
• Syncs sessions → Express backend	• Syncs sessions → same Express backend
• Authenticates with same JWT	• Authenticates with same JWT

Bullets:

- **Chrome Extension** — Manifest V3, background service worker timer, `declarativeNetRequest` site blocker, syncs sessions to the leaderboard with the same JWT as the web app
- **GNOME Extension (Tamatem)** — Native Linux desktop timer in the system top bar, OS-level site blocking, session sync — no browser tab needed at all
- Both are independent clients of the same Express API, with the same authentication

Speaker notes: "We did not stop at the website."

We built a Chrome extension that runs the Pomodoro timer in a background service worker — a background process that Chrome keeps alive even when the popup is closed. During focus sessions, the extension activates blocking rules using Chrome's `declarativeNetRequest` API — sites like YouTube are blocked at the HTTP request layer, not through a JavaScript trick you can bypass. The session is logged to our backend and appears on the leaderboard.

But more unusually, we also built a GNOME Shell extension called Tamatem. GNOME is the desktop environment on Ubuntu, Fedora, and most Linux distributions — the systems that many computer science students use as their primary development environment. Tamatem puts a live Pomodoro timer directly in the Linux system top bar. It blocks sites at the operating system level. It syncs sessions to the same Cortex backend through the same API.

Think carefully about what this means: a student can use the Cortex web app on any device, the Chrome extension in their browser, or the GNOME extension on their Linux desktop — and all three share the same notes, the same leaderboard, the same data. One login. One backend. One system."

 **بالعربي:** «احنا ما وقفناش عند ال

بنينا Chrome extension بيشغل ال Pomodoro timer جوا — background service worker ال Chrome process بيخليه شغال حتى لو أقفلت ال popup. أثناء ال focus sessions، ال extension بيفعل blocking rules باستخدام Chrome's `declarativeNetRequest` API — مواقع زي YouTube بتتجب على مستوى ال HTTP request، مش بحيلة JavaScript ممكن تتفادها. ال session بتتسجل على ال backend وبتظهر على ال leaderboard.

بس الأكثر تميزاً، احنا كمان بنينا GNOME Shell extension اسمه GNOME Tamatem. هو ال desktop environment على Ubuntu و Fedora وغالبية توزيعات Linux — اللي كتير من طلبة CS بيستخدموها ك بيئة تطوير رئيسية. Tamatem بيحط Pomodoro timer حي ومباشر في ال Linux system top bar. بيحجب المواقع على مستوى نظام التشغيل. وبيـsync ال sessions لل backend بتاع Cortex نفسه.

فكر كويس في معنى ده: الطالب ممكن يستخدم الـ Cortex web app على أي device، أو Chrome extension في الـ browser، أو GNOME extension على الـ Linux desktop — والتلاتة بيشاركوا نفس الـ notes، ونفس الـ leaderboard، ونفس الـ data. Login واحد، Backend واحد، System واحد.»

Timing: 2 minutes **Transition:** "Let me walk through the product, system by system."

SLIDE 5 — Product: The Notes System

Title: Notes — A Full Academic Writing Environment

Bullets:

- **Rich text editor** — headings, tables, code blocks with syntax highlighting, images, checklists, embeds
- **Slash commands** — type `/` to insert any block type; keyboard-first, no mouse required
- **/ai command** — AI writes, edits, or comments inline, streaming character-by-character
- **Organization** — nested folders + color-coded tags + fast filter bar
- **Collaboration** — share with a teammate as viewer or editor; publish publicly for community access
- **AI sidebar** — Summarize the note · Suggest tags · Ask the Library Assistant
- **Archive system** — soft delete with restore, or permanent delete

Speaker notes: "The notes system is the heart of Cortex. The editor is built on Plate.js — a structured rich-text framework built on top of Slate.js. It produces documents as structured JSON, not raw HTML. Every block — a heading, a paragraph, a code block — is a node in a JSON tree. This choice was not aesthetic; it was architectural. The backend can walk this JSON tree to extract clean text for AI processing, something that would be fragile and unreliable with raw HTML.

Slash commands make the editor keyboard-native. Type `/table` to insert a table, `/code` to insert a code block, `/ai` to open the AI command panel.

The AI command is embedded in the editor itself. Type `/ai`, write a prompt — 'explain this concept using an analogy' — and the AI response streams into your document word by word, in place, without leaving the editor.

Notes are organized with folders nested inside folders, and color-coded tags. You can share any note with another student — viewer-only or full editor access. You can

publish any note publicly, generating a URL that anyone can open without logging in. This turns student notes into community study resources."

🇪🇬 **بالعربي:** «ال notes system ده قلب Cortex. ال editor بُني على Plate.js — rich-text framework مبني فوق Slate.js. documents ك structured JSON، كل HTML مش الاختيار ده ما كانش. JSON tree في node هو — block — heading, paragraph, code block — حاجة AI، ال plain text ده ويطلع منه JSON tree يقدر يمشي على ال backend جمالي، كان معماري. ال HTML كانت هتبقى هشة وغير موثوقة لو استخدمنا

ال slash commands بتخلي ال editor keyboard-native. اكتب `/table` تدرج `/code`، table تدرج `/ai` code block تفتح ال AI command panel.

ال AI مدمج في ال editor نفسه. اكتب `/ai`، اكتب prompt — 'اشرح المفهوم ده بمثال' — وال AI response بيـ stream في ال document كلمة كلمة، في مكانه، من غير ما تعمل أي tab switch.

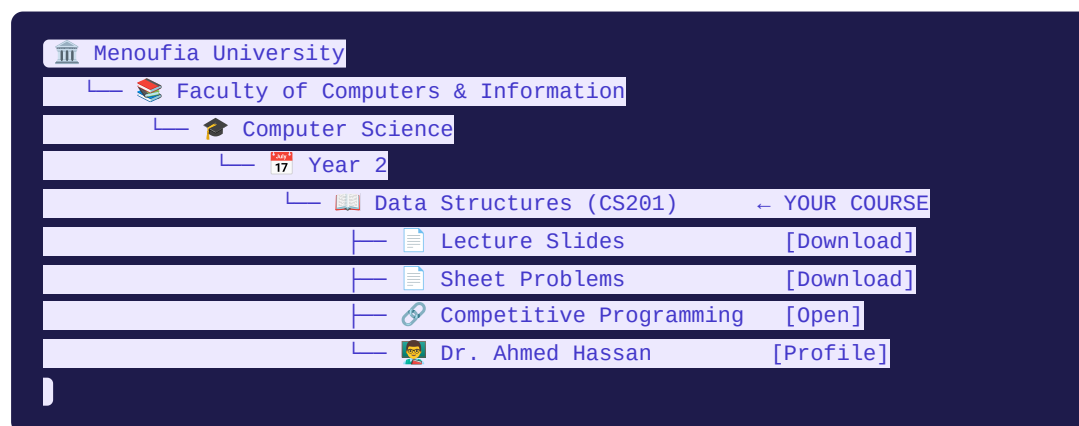
ال notes منظمة بـ folders جوا folders، و color-coded tags. تقدر تشارك أي note مع زميل — view فقط أو edit كامل. وتقدر تنشر أي note للعموم، وبتولد URL أي حد يفتحه من غير ما يسجل دخول. ده بيحول notes الطلبة لـ resources مجتمعية للمذاكرة.»

Timing: 1 min 30 sec

SLIDE 6 — Product: The Resource Registry

Title: The University Catalog — One Home for All Academic Materials

Visual — Hierarchy tree:



Bullets:

- Organized by: University → College → Major → Year → Course
- Resources per course: PDFs, external links, videos — managed by admins
- Doctor assignments: who teaches which section

- **Auto-filtered to YOUR curriculum** — your courses front and center when you log in
- Bilingual: every entity has an Arabic name and an English name
- Admin-managed: only designated admins can add or edit catalog content

Speaker notes: "The Resource Registry solves the WhatsApp problem. It is a structured, admin-managed catalog of the university's complete academic content — following the real hierarchy of Egyptian universities: university, college, major, year level, course.

The moment you complete your profile setup — selecting your university, college, and major — the catalog opens pre-filtered to your exact curriculum. You do not scroll through hundreds of other departments. Your courses are front and center.

Each course page shows the resources available for download — PDFs, lecture videos, reference links — and the doctors who teach it. Admins upload the resources. Students browse and download. Simple, structured, permanent — no expiring WhatsApp messages."

🇪🇬 **بالعربي:** «ال Resource Registry حل مشكلة ال WhatsApp ده. ده catalog و admin-منظم. لكل المحتوى الأكاديمي في الجامعة — بيتبع الهرم الحقيقي للجامعات المصرية: جامعة، كلية، course، تخصص، سنة.

في اللحظة اللي بتخلص فيه ال profile setup بتاعك — بتختار الجامعة، الكلية، والتخصص — ال catalog يفتح مفلتر مسبقاً على ال curriculum بتاعك بالظبط. مش بتا scroll في مئات الأقسام الثانية. ال courses بتاعتك هي اللي بتظهر أول حاجة.

كل صفحة course بتبين ال resources المتاحة للتحميل — PDFs، فيديوهات المحاضرات، links مرجعية — والدكاترة اللي بيدرسوها. ال admins بيرفعوا ال resources. الطلبة بيتصفحوا ويحملوا. بسيطة، منظمة، دائمة — من غير رسائل WhatsApp بتنتهي.»

Timing: 1 min 15 sec

SLIDE 7 — Product: The Daily Journal & Calendar

Title: The Academic Diary — A Day-by-Day Record of Your Learning

Visual — Calendar + day view:

JULY 2026						
Mo	Tu	We	Th	Fr	Sa	Su
		1	2	3	4	5
6	7	8	[•]	[•]	[•]	12 ← dots = days with a log entry
13	14	15	16	17	18	19

TODAY – Thursday, July 9:

🌟 Highlight: "Finally understood virtual memory paging!"

📝 Note: [Rich text – what I studied, links to notes, code]

✅ Tasks:

[✓] Review OS Chapter 5 – paging & TLB

[✓] Solve 2 algorithm problems

[] Read AI lecture notes before tomorrow

Bullets:

- One structured page per day — created on demand when you click a date
- **Highlight field** — your best achievement today (builds journaling habit)
- **Rich text note** — same Plate.js editor as the notes system
- **Task list** with instant checkbox response (optimistic UI — no waiting)
- **Calendar view** — dots mark every day you have written an entry
- Every log is AI-embedded — later searchable by meaning via the Library Assistant

Speaker notes: "The daily planner gives each day a structured workspace. When you start studying, you open today's page, write what you plan to work on, and add your tasks. At the end of the session, you record your highlight — the best thing you understood or accomplished.

The calendar makes your study history visible across the entire semester. Every day with an entry has a dot. Click any past day and you see exactly what you wrote, what you completed, what you were thinking. Over a semester, this becomes an invaluable record of your academic journey.

And here is the architectural connection to the AI system: every time you save a daily log, the backend runs an embedding pipeline asynchronously. It converts the log's content into a 768-dimensional vector and stores it in the database. Later, when you ask the Library Assistant 'what was I studying last week when I was confused about scheduling?' — it searches these vectors. It finds the relevant day not by matching keywords, but by understanding meaning."

منظمة. لما تبدأ تذاكر، بتفتح صفحة النهارده، workspace بيدي كل يوم **بالعربي:** «الـ daily planner أحسن — highlight في آخر الجلسة، بتسجل الـ tasks بتكتب إيه اللي ناوي تشتغل عليه، وبتضيف الـ حاجة فهمتها أو أنجزتها.

الـ calendar بيخلي تاريخ مذاكرتك مرئي على مدار الترم كله. كل يوم فيه entry ليه نقطة. اضغط على أي يوم فات وهتشوف بالظبط إيه اللي كتبتة، إيه اللي خلصته، إيه اللي كنت بتفكر فيه. على مدار ترم، ده بيبقى سجل قيم جداً لرحلتك الأكاديمية.

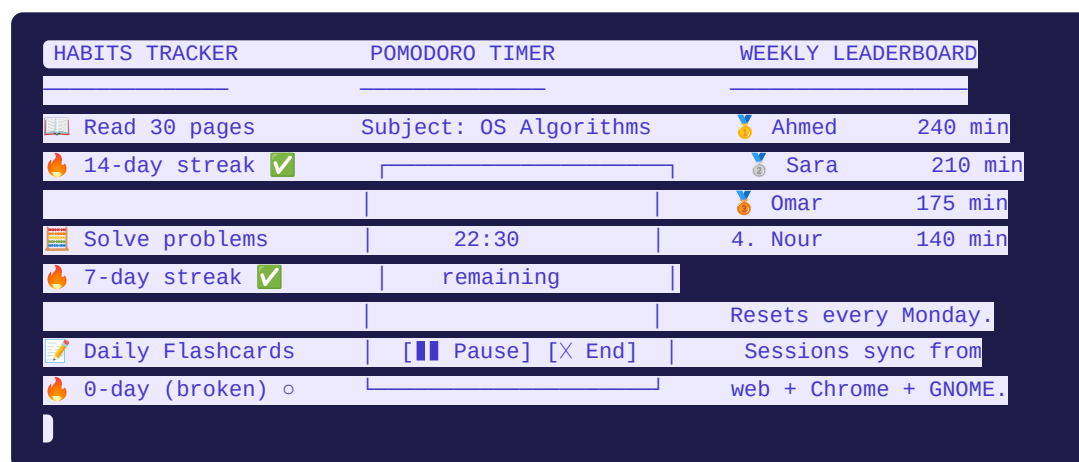
والربط المعماري بالـ AI system: كل مرة بتسيّف daily log، الـ backend بيشتغل embedding
pipeline بشكل asynchronous. بيحوّل محتوى الـ log لـ vector من 768 رقم ويبخّطه في الـ
database. بعدين لما تسأل الـ Library Assistant 'أنا كنت بذاكر إيه الأسبوع اللي فات وأنا مش فاهم
scheduling؟' — بيدور في الـ vectors دي. بيلاقي اليوم المناسب مش بمقارنة الكلمات، لكن بفهم
المعنى.»

Timing: 1 min 30 sec

SLIDE 8 — Product: Habits, Pomodoro & Social Leaderboard

Title: Productivity + Psychology + Accountability

Visual — Three-panel layout:



Bullets:

- **Habits** — daily, weekly, or monthly with custom frequency; streak tracking with visual indicators
- **Pomodoro** — 25 min focus / 5 min break / 15 min long break; subject-labeled; every session logged
- **Friends & Study Groups** — add friends, form study groups, start shared focus sessions
- **Weekly Leaderboard** — friends ranked by total focus minutes; resets every Monday
- **Cross-client sync** — Chrome extension and GNOME extension sessions appear on the same leaderboard

Speaker notes: "The productivity layer has three interlocking parts, each supported by a psychological principle."

Habits use the streak mechanism — every consecutive day you complete a habit, the streak counter goes up. Breaking a 21-day streak feels genuinely painful. That pain is not a bug — it is the psychology that keeps habits alive.

The Pomodoro timer is built on decades of research: 25 minutes of focused, uninterrupted work, followed by a 5-minute break. After four cycles, a longer 15-minute break. Every completed session is logged to the database with a timestamp and the subject being studied. This data feeds the leaderboard.

The social layer is what makes the system sticky. You add friends. You see that a friend has studied 240 minutes this week while you have only done 140. That number motivates you. The leaderboard resets every Monday — last week's winner must earn it again. Study groups let teammates start shared sessions together.

And critically: whether a session is logged from the web app, the Chrome extension, or the GNOME extension — it flows through the same Express backend to the same database. One leaderboard, three clients."

فيهِ ثلاث أجزاء متشابهة مع بعض، كل واحدة بتعتمد على مبدأ productivity layer **بالعربي:** «الـ  نفسي.

الـ habits بتستخدم آلية الـ streak — كل يوم متتالي بتعمل فيه الـ habit، الـ counter بيعلى. كسر streak من 21 يوم بيبقى موجه فعلاً. الـ الـ bug — ده هو علم النفس اللي بيخلي العادات تتمسك.

الـ Pomodoro timer مبني على أبحاث بتاعة عقود: 25 دقيقة focus عميق بدون مقاطعة، وبعدين استراحة 5 دقائق. بعد 4 cycles، استراحة طويلة 15 دقيقة. كل session مكتملة بتتسجل في الـ database بـ timestamp والـ subject اللي بتذاكره. الـ data دي بتغذي الـ leaderboard.

الـ social layer هو اللي بيخلي الـ system يلصق. بتضيف أصحابك، بتشوف إن صاحبك ذاكر 240 دقيقة الأسبوع ده وانت عملت 140 بس. الرقم ده بيحتمسك. الـ leaderboard بيـresets كل يوم اثنين — اللي كسب الأسبوع اللي فات لازم يكسبه تاني.

والنقطة المهمة: سواء الـ session اتسجلت من الـ web app، أو Chrome extension، أو GNOME extension — كلها بتوصل لـ Express backend نفسه و database نفسه. Leaderboard واحد، ثلاث clients.

Timing: 1 min 45 sec

SLIDE 9 — The Architecture: How Everything Connects

Title: Three Clients. One Backend. One Rule.

Visual — Architecture diagram:



Speaker notes: "The architecture we designed has one rule that governs everything, and it cannot be violated:

No client — not the web app, not the Chrome extension, not the GNOME extension — ever talks to the database directly. Everything passes through the Express backend.

This matters for a specific reason. Supabase has a service role key — an admin credential that bypasses all security policies and can read and write any data in the database. That key must never touch a client environment. It lives exclusively on the Railway server. Even if there is a bug in the frontend, even if an attacker inspects every browser request — they will never find this key.

The backend is organized in three strict layers for every domain: Controller handles HTTP parsing and response formatting. Service handles business logic and cross-

service orchestration. Repository handles all database calls — no Supabase query ever appears outside a Repository file.

Seven domains follow this pattern: Auth, Notes, Daily, Data, AI, Admin, Workspace.
Same structure. Always.

And every client — three different clients, three different platforms — calls the same Express API with the same JWT token. One backend serves all of them equally. This is the architectural backbone that made everything else possible."

واحدة بتحكم كل حاجة، وما ينفعش تتكسر rule اللي صممناه فيه architecture بالعربي: «ال

client — لا ال web app، ولا ال Chrome extension، ولا ال GNOME extension — يتكلم مع ال database مباشرة. كل حاجة بتعدي على ال Express backend.

ده مهم لسبب محدد. Supabase عنده credential — service role key — تدميني بيتجاوز كل ال security policies وممكن يقرأ ويكتب أي data في ال database. المفتاح ده ما ينفعش يلمس أي بيئة client. بيعيش حصراً على ال Railway server. حتى لو فيه bug في ال frontend، حتى لو مهاجم راجع كل request في ال browser — ما هيلاقيش المفتاح ده أبداً.

ال backend منظم في ثلاث طبقات صارمة لكل domain: ال Controller بيتعامل مع ال HTTP parsing وتنسيق ال response. ال Service بيتعامل مع ال business logic والتنسيق. ال Repository بيتعامل مع كل ال database calls — ما فيش Supabase query بيظهر خارج ملف Repository أبداً.

سبع domains بتتبع ال pattern ده: Auth، Notes، Daily، Data، AI، Admin، Workspace. نفس الهيكل. دائماً.

وكل client — ثلاث clients مختلفة، ثلاث platforms مختلفة — بيكلموا نفس ال Express API بنفس ال JWT token. Backend واحد بيخدمهم الثلاثة بالتساوي. ده هو ال backbone المعماري اللي خلى كل حاجة تانية ممكنة.»

Timing: 1 min 45 sec **Transition:** "Now let me explain why we chose each technology."

SLIDE 10 — Technology Decisions: None Were Defaults

Title: Every Technology Choice Was Deliberate

Visual — Decision table:

We chose	Over	The reason
Plate.js	Quill, TipTap	Plate.js produces structured JSON — the backend walks this tree for AI processing. Quill produces HTML — brittle to parse.

Google Gemini	OpenAI, Anthropic	Best Arabic quality at this price tier. \$0.075/M tokens vs \$10–30/M for GPT-4. Only provider with both quality chat AND embedding.
Supabase	Firebase, raw PostgreSQL	RLS + pgvector + managed Auth in one service. Firebase is NoSQL — our relational schema required PostgreSQL.
Next.js 15	Vite SPA	Server Components render the catalog instantly on the server. Admin guards run server-side — no JavaScript to bypass.
Separate Express backend	Next.js API routes only	Service role key must be isolated from Vercel. Three clients need one shared API that is not a frontend implementation detail.
declarativeNetRequest	Content script blocking	Blocks at the HTTP network layer — cannot be bypassed, works in Manifest V3, approved by Chrome policies.
GNOME Shell extension	Nothing	Extends Cortex to the Linux desktop — persistent timer in the system tray, no browser tab required.

Speaker notes: "None of these were defaults. Every one was a deliberate choice made after evaluating the alternative.

The most consequential choice was Plate.js over Quill. Plate.js produces a structured JSON tree — every paragraph, every heading, every code block is a typed node with children. The backend can walk this tree, extract clean text, and feed it to the AI pipeline without any parsing. Quill produces HTML. We would have needed to strip tags and parse HTML every time we wanted to run an AI operation on a note. Fragile, slow, and error-prone.

The second most consequential: Gemini over OpenAI. Better Arabic quality — we tested all providers with academic Arabic text. Dramatically cheaper — 100 to 400 times cheaper than GPT-4 at comparable quality. And critically: it is the only major provider that gives us both a quality chat model and a quality embedding model in one API. Anthropic has no embedding model at all.

The separate Express backend was not complexity for its own sake. It was a security requirement: the service role key must live on infrastructure completely separate from the frontend deployment. And it was a product requirement: three clients need one API.

That API should not be a Next.js implementation detail buried in the frontend repository."

كل واحدة كانت اختيار مدروس بعد ما قيّمنا البديل. default بالعربي: «ما فيش واحدة من دول كانت

الاختيار الأكثر تأثيراً كان Plate.js بدل Plate.js. Quill. JSON tree منظم — كل paragraph. كل heading, كل code block هو node بـ type و children. الـ backend يقدر يمشي على الـ tree ده، يطالع منه plain text، ويديّه لـ AI pipeline من غير أي parsing. Quill. HTML. كنا هنحتاج نشيل الـ tags ونـ parse الـ HTML كل مرة عايزين نعمل AI operation على note. هش، بطيء، وعرضة للأخطاء.

الاختيار الثاني الأكثر تأثيراً: Gemini بدل OpenAI. جودة عربي أفضل — اتجربناهم كلهم مع نصوص عربية أكاديمية. أرخص بكثير — من 100 لـ 400 ضعف أرخص من GPT-4 عند نفس الجودة. والأهم: ده الـ provider الوحيد اللي بيدّينا chat model كويس و embedding model كويس في نفس الـ API. Anthropic ما عندوش embedding model خالص.

الـ Express backend المنفصل ما كانش تعقيد لمجرد التعقيد. كان متطلب أمني: الـ service role key لازم يعيش على infrastructure منفصل تماماً عن الـ Vercel deployment. وكان متطلب معماري: ثلاث clients محتاجين API واحد. الـ API ده ما ينفعش يبقى implementation detail جوّا الـ frontend repository».

Timing: 2 minutes

SLIDE 11 — The Scale of What We Built

Title: Engineering Scope — Cortex by the Numbers

Visual — Metrics table:

Component	Scope
Database migrations	44 SQL files — tables, indexes, RLS policies, pgvector schema, triggers, RPC functions
Backend service domains	7 (Auth, Notes, Daily, Data, AI, Admin, Workspace) — each with Controller + Service + Repository
API endpoints	50+ — all with auth guards, validation, and error handling
Frontend routes	15+ — App Router, Server Components, admin guards, bilingual layouts
Plate.js editor plugins	20+ — slash commands, AI command, copilot, tables, images, code blocks

AI systems	3 — Editor AI (streaming), Note AI (local NLP), Library Assistant (RAG + pgvector)
AI model providers	4 supported — OpenAI, Anthropic, Groq, Google
Chrome Extension	5-tab popup, service worker timer, declarativeNetRequest blocker, backend sync
GNOME Extension (Tamatem)	Top bar timer, OS-level site blocking, session sync via Cortex API
Internationalization	Arabic + English, RTL CSS logical properties, locale switcher, 200+ translated keys
Deployment	Vercel + Railway + Supabase — environment isolation, CI/CD, health checks
Lines of TypeScript	15,000+

Speaker notes: "This table represents the engineering scope of the system we built together as a team."

44 SQL migrations — not because we redesigned constantly, but because we built iteratively and correctly, migrating the schema precisely at each step. 50 API endpoints. Three clients. Three AI subsystems.

Each member of this team owned one or more subsystems within this architecture. My role was to design and maintain the shared structure that allowed all of them to build simultaneously without conflicts, and to build the core infrastructure: the database layer, the backend architecture, the deployment pipeline, and the AI system.

The six presentations that follow will go deep into each subsystem. I will be available to answer any questions about the overall design and system integration — now or after the session."

team. الی بنیانا مع بعض ك system بالعربی: «الجدول ده ییمل الحجم الهندسى لل»

مش لأننا كنا بنعيد التصميم باستمرار، لكن لأننا كنا بنبني بشكل تدريجي وصح، 44 SQL migration —
 50 migrate the schema. ثلاث clients. ثلاث API endpoints. كل خطوة. 50 migrate the schema. ثلاث clients. ثلاث subsystems.

كل فرد في الـ team يمتلك subsystem واحد أو أكثر في الـ architecture دي. دوري كان إني أصمم وأحافظ على الهيكل المشترك اللي خلاهم بينوا في نفس الوقت من غير تعارض، وإني أبني الـ core infrastructure: الـ database layer، الـ backend architecture، الـ deployment pipeline، والـ AI system.

الست presentations اللي جاية هتتعمق في كل subsystem. أنا متاح للرد على أي أسئلة عن ال overall design وتكامل ال system — دلوقتي أو بعد الجلسة.»

Timing: 1 minute

SLIDE 12 — The Handoff

Title: What Follows — Six Deep Dives

Visual — Presentation order:

Section 1 ← YOU ARE HERE	Architecture, Vision, Technology Decisions
Section 2	Authentication, Session Management & User Profiles
Section 3	Notes System – Rich Text Editor & Collaboration
Section 4	AI Integration – Embeddings, RAG & Streaming
Section 5	University Resource Catalog & Data Management
Section 6	Daily Planning, Habits, Pomodoro & Social Leaderboards
Section 7	Admin Panel, Arabic/English, Chrome & GNOME Extensions

Speaker notes: "Everything that follows is a deep dive into the subsystems my teammates implemented within this architecture.

Each of them will show you the code, the schemas, the data flows, and the engineering decisions specific to their domain. The architecture you heard from me is the foundation underneath all of them.

I am happy to take questions about the overall system design, the architectural decisions, or anything I have presented. Otherwise, I will hand over to the next presenter."

(Pause. Smile. Stand confidently.) "Thank you."

architecture اللي زمايلي طوّروها جّوا ال subsystems **بالعربي:** «كل اللي جاي بعدي هو تعمق في ال  دي.

كل واحد فيهم هيبين ال code، وال schemas، وال data flows، والقرارات الهندسية الخاصة ب domain بتاعه. ال architecture اللي سمعته مني هو الأساس اللي تحتهم كلهم.

أنا سعيد بالرد على أي أسئلة عن ال overall system design أو القرارات المعمارية أو أي حاجة قدّمته. وغير كده، هسلّم لل presenter الجاي.»

(وقفة. ابتسامة. وقفة واثقة.) «شكراً.»

Timing: 45 seconds

Q&A Preparation — Full Answer Bank

Q: What is the GNOME extension and why did you build it? A: GNOME is the desktop environment used by Ubuntu, Fedora, and most Linux distributions — the systems that computer science students commonly run. A GNOME Shell extension integrates directly into the desktop's system bar. We built Tamatem (tamatem@frey.dev) so Linux users — especially CS students on Linux — get a persistent Pomodoro timer in their system tray and site blocking at the OS level, with no browser tab required. It connects to the same Express API with the same JWT authentication as the web app and Chrome extension.

Linux وغالبية توزيعات Fedora و Ubuntu على desktop environment هو ال GNOME **بالعربي** — يتكامل مباشرة في GNOME Shell extension. يشغلونها عليها CS اللي كتير من طلبية ال systems ال — Pomodoro timer يكون عندهم Linux عشان طلبية Tamatem بنينا. desktop. بتاع ال system bar ال browser على مستوى نظام التشغيل، من غير ما يفتحوا أي site blocking و system tray دايم في ال Chrome وال web app اللي بيستخدمه ال JWT بنفس ال Express API بيتوصل بنفس ال tab. extension.

Q: Why build both a Chrome extension AND a GNOME extension? A: Different students use different environments. A Windows or macOS student uses the Chrome extension. A Linux student running GNOME gets native OS integration through Tamatem. Both clients hit the same API, so their sessions appear on the same leaderboard regardless of which interface they used. We wanted Cortex to feel native on every platform, not just tolerable.

يستخدم الـ macOS أو Windows مختلفة. طالب على environments بالعربي: الطلبة يستخدمون
من خلال native integration يتحصّل على GNOME مع Linux طالب على Chrome extension.
بتاعتهم بتظهر على نفس الـ sessions فالـ API، بيكلّموا نفس الـ clients الاتنين Tamatem.
platform، على كل native يبقى Cortex المستخدم. أردنا interface بغض النظر عن الـ leaderboard
مش مجرد شغال.

Q: Why a separate Express backend instead of Next.js API routes? A: Two reasons. First — security: the database service role key must live on infrastructure completely separate from the Vercel deployment. Next.js API routes run on Vercel, alongside the frontend — that creates key exposure risk. Second — architecture: three separate clients (web, Chrome, GNOME) need one shared API. That API should not be a Next.js implementation detail. It is a first-class service.

لأهم شيء، لا يزال يتعين على database بتتبع الـ service role key **بالعربي**: سببنا الأول — الأمان: الـ **ال** Vercel بتتغل على الـ Next.js API routes. الـ Vercel منفصل تماماً عن الـ infrastructure API منفصلين محتاجين الـ clients الثاني — المعمارية: ثلاث key. ده بيخلق خطر تسريب للـ frontend من service هو — Next.js جوا implementation detail ده ما ينفعش يبقى API واحد مشترك. الـ الدرجة الأولى.

Q: How does data sync work across three different clients? A: All three authenticate with the same JWT from Supabase Auth. The web app stores it in an httpOnly cookie. The Chrome extension stores it in `chrome.storage.local`. The GNOME extension stores it in GNOME's settings schema. All three make HTTP calls to the same Railway Express backend with `Authorization: Bearer <token>`. The backend verifies the token and processes the request identically, regardless of which client is calling.

ال بيحطه في الـ web app من Supabase Auth. الـ JWT **بالعربي**: الثلاثة بيتعمدوا على نفس الـ **ال** httpOnly cookie. الـ GNOME **وال** `chrome.storage.local` بيحطه في الـ Chrome extension. الـ GNOME extension بيحطه في الـ GNOME's settings schema. الـ Railway Express backend HTTP calls الثلاثة بيعملوا. الـ backend verify الـ token نفسه مع `Authorization: Bearer <token>`. الـ client بنفس الطريقة بغض النظر عن انهو request الـ process.

Q: What was the hardest technical challenge? A: The AI embedding pipeline. Getting pgvector configured correctly, choosing the right embedding model (`text-embedding-004`), tuning the cosine similarity threshold so the Library Assistant retrieves relevant notes without false positives, and making embedding run asynchronously after every save without blocking the user experience — all of this required significant iteration. The final solution runs `syncLogEmbedding` in the background and fails silently, so the save operation always feels instant to the user.

ال AI embedding pipeline. الـ pgvector إنا نضبط الـ embedding model صح، نختار الـ Library Assistant عشان الـ cosine similarity threshold نضبط الـ (`text-embedding-004`) المناسب. الـ embedding يشتغل وإنا نخلي الـ false positives المناسبة من غير notes يجيب الـ Assistant iteration كل ده محتاج — user experience من غير ما يبطئ الـ save بعد كل asynchronously الـ save ولو فشل بيفشل بهدوء، فعملية الـ background في الـ `syncLogEmbedding` الحل النهائي بيشغل instant. دائماً بتحس إنها.

Q: Why Supabase over raw PostgreSQL or Firebase? A: Three capabilities we needed that Supabase packages together. Row-Level Security — policies that execute at the database level, so even a buggy query cannot return data it should not. `pgvector` — the PostgreSQL extension for 768-dimensional vector similarity search, which powers

the entire Library Assistant. And managed Auth — JWT issuance, refresh token rotation, and SSR cookie helpers, handled without custom implementation. Firebase is a NoSQL document database — our relational schema with 44 migrations and complex joins is fundamentally incompatible with Firebase's data model.

🇸🇪 Row-Level Security — policies. Supabase بالعربي: ثلاث قدرات احتجناها ما ينفعش ترجمه data مغلوطة ما بترجعش query فحتى، database، بتنقذ على مستوى الـ اللي بيشتغل الـ dimension، بـ 768 vector similarity search الـ PostgreSQL امتداد — `pgvector` Library Assistant و refresh token وتدوير الـ JWT إصدار الـ Managed Auth كله. ومع ذلك Library Assistant هو NoSQL document database — Firebase. كده جاهرة من غير إننا نكتب حاجة. Firebase هو NoSQL document database — Firebase. مش متوافق أصلاً مع نموذج بيانات complex joins و migration بتاعنا مع 44 relational schema الـ Firebase.

Q: How does Arabic RTL layout work technically? A: When the user switches to Arabic, `dir="rtl"` is set on the HTML element. This causes the browser to reverse the layout direction. We use CSS logical properties throughout the application — `padding-inline-start` instead of `padding-left`, `margin-inline-end` instead of `margin-right` — so the layout mirrors correctly without Arabic-specific CSS overrides. The catalog schema stores `name_en` and `name_ar` for every entity, so content is also bilingual, not just the interface.

🇸🇪 ده بيخلي الـ HTML element على الـ `dir="rtl"` بالعربي: لما المستخدم يبدل للعربي، بيتحدد browser application — CSS logical properties بنستخدم. layout يعكس اتجاه الـ application — `padding-left` بدل `padding-inline-start`، `margin-right` بدل `margin-inline-end`، `padding-left` بدل `padding-inline-start` — `name_en` بيخزن catalog schema خاص بالعربي. الـ CSS بينعكس صح من غير ما نكتب layout فال interface مش بس الـ، bilingual كمان content فال، entity لكل و `name_ar`.

Q: What does "44 database migrations" mean? A: Migrations are versioned SQL files that define the database schema incrementally. Each migration adds or modifies tables, indexes, policies, or functions. 44 migrations means the schema evolved through 44 deliberate, version-controlled steps — every table, every foreign key, every RLS policy, every pgvector index, every SQL function is explicitly defined and reproducible. This is standard production database engineering practice.

🇸🇪 بشكل تدريجي. database schema متسلسلة بتعرف الـ SQL هي ملفات migrations بالعربي: الـ 44 migration يعني الـ tables أو indexes أو policies أو functions. 44 migration بتضيف أو تعدل migration كل table، كل foreign key، كل RLS policy، كل pgvector index، كل SQL function، دي الممارسة للتكرار. الـ production databases الهندسية الاحترافية المعيارية في الـ

Q: Can the platform scale beyond a single university? A: Yes — the hierarchical catalog schema was designed for this. The `universities` table is the root. Each university has colleges, each college has majors, each major has year levels, and each year level has courses. Adding a second university means inserting a new row in the `universities` table and populating its hierarchy. The filtering logic is already university-aware.

اتصمّم لده من الأساس. جدول hierarchical catalog schema بالعربي: أيوه — ال  كل جامعة عندها كليات، كل كلية عندها تخصصات، كل تخصص عنده root. هو ال `universities` جديدة في جدول row إضافة جامعة ثانية معناها إننا نضيف courses سنوات، وكل سنة عندها أصلاً بيعمل بوعي بالجامعة filtering logic ونملي الهرم بتاعها. ال `universities`
